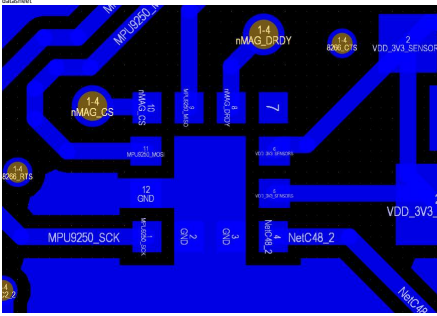
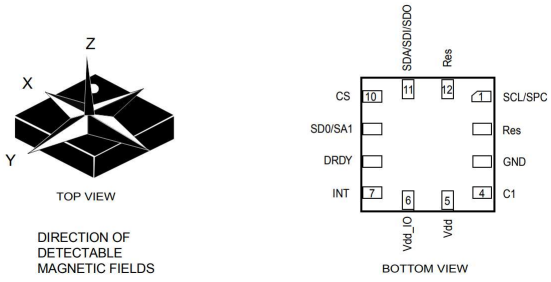
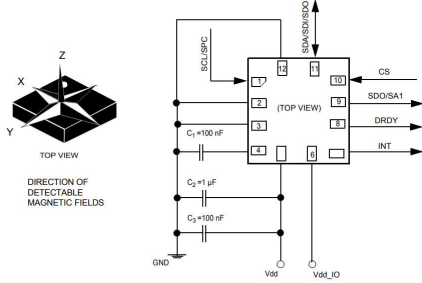


Figure 2. Pin description



current config?? Why inverted?? Need to look into it, wiring seems correct but the part physically won't work with this footprint

Figure 5. LIS3MDL electrical connections



This makes more sense, they went off application config
First image was bottom view, problem solved!

2.2-Order PCB and parts

order put through for icm-42688 and switches along with v2 board. Fingers crossed the board has not many issues(it did)

JOSHUA_HECKE_WIL

EE&I Order Template v3

Name

Joshua Hecke

Email

josh_hecke@aut.edu.au

Account

524530/4111 (EE&I Consumables)

Supervisor

Brendan Storch

Unit

QUT AutoPilot Research

Group

WVL

Multiplier

1

Do not leave empty cells in the descriptions.

Do not change formatting or file type.

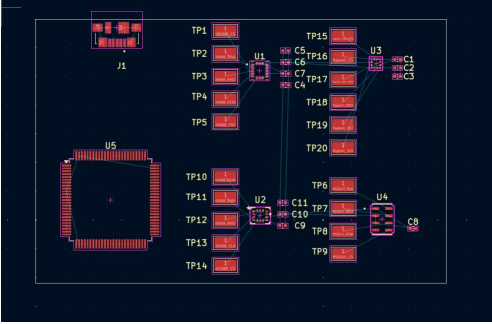
Complete this form and e-mail to fox.ave@qut.edu.au

Also cc your supervisor who will authorise the order

Item	Description	Quantity	Cost Each	Currency	Supplier	Order Code	Web Link
1	TOK Inverness KM-42688-P	5	\$4.30	USD	Dasenic	8888-P-DS	https://www.dasenic.com/product/tok-inverness-km-42688-p-88888834
2	C&C PT3830 SR 250-3MTR LFS	6	\$0.31	USD	Dasenic	3MTR LFS-DS	https://www.dasenic.com/product/c&c-pt3830-sr-250-smt-lfs-7515346
3	PCBWAY Order 10215228252401 • Stereo	5		USD	PCBWAY	112165228252401	
4							
5							
6							
7							
8							
9							
10							
11							
12							
13							
14							
15							
16							
17							
18							
19							
20							
21							
22							
23							
24							
25							
26							
27							
28							
29							
30							
31							
32							
33							
34							
35							
36							
37							
38							
39							
40							
41							
42							
43							
44							
45							
46							
47							
48							
49							
50							

Order issues so fixing them up now (PCBWAY email rep, im thinking its drill hole settings that were not communicated properly)

Mean time working on designing a test board for the sensors and the MCU so the smd sensors can be tested solo with the bare min parts then integrated all together later. This is a mini project with the main goal is to improve circuit design and datasheet reading skills while main design is being fabricated. Key hardships are foot print selection and considering key choices like how am I flashing code to this board, how am I powering it? How to wire up a MCU like the STM? Done in Kicad



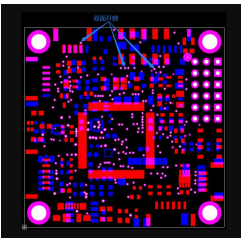
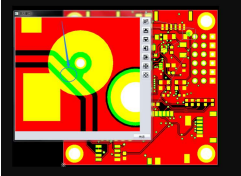
need to add MCU connection and button to finish design. Consider what parts are needed for usb power to board power connection. Goal is to test spi comms with all sensors, then use success to build main board

22/12/2025-

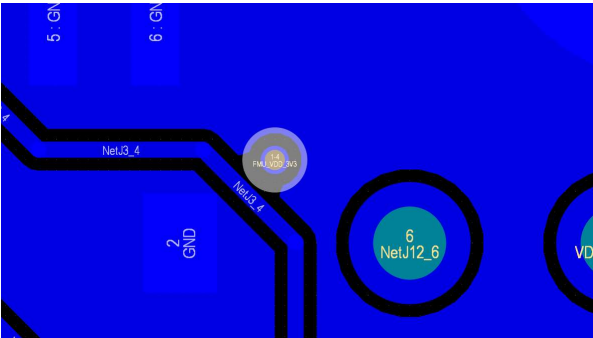
PCB failed during review before manufacturing & ,here is output from PCB techs.

Feedback from PCBway:

W54323453401 you ordered is tenting vias but the file shows vias not covered pls advise if we review follow the file. The file includes exposed traces,pls advise if the file is correct



FXS1 - fix exposed tracer

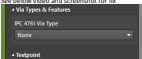


FXS2 - tented via is the process of covering vias so they are protected from corrosion,dust etc. see

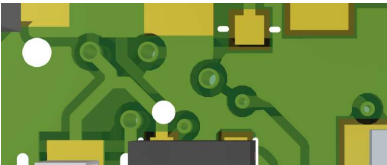
he idea behind via tenting is simple: you're covering any vias in your PCB with solder mask so that any pad/ring on the via hole, and the via barrel itself, are not exposed to the environment. The solder resist is placed to provide some measure of protection for the via pad and the plating inside the via barrel. If you look at a PCB layout, you can spot tented vias just by looking at your solder stop mask layer; the same applies to the Gerber file for that solder mask layer.

From: <https://electronics.stackexchange.com/questions/449616/via-tenting-vs-plating>

See below video and screenshot for fix



<https://www.youtube.com/watch?v=43TLuGyPmU>



Above you can see vias are now covered and not exposed

The new files contains the files required, sort off for evaluation by manufacturer, more progress on KICAD testar board done, added buttons and a required regulator to get the needed 3.3 volts. Need to now wire STM correct and connect SPI lines. It's a 4 layer board so this is practice with routing techniques ensure its tested.

New pcb has been ordered

23/12/2025-

Issues with pcb manufacturing again, below is the output

HL

About the pcb order W54323453404, there are 3 questions need to confirm:

1. For the board thickness to hole diameter ratio is greater than 10:1, it is recommended to enlarge the hole size,pls revise the file.

2. Pls confirm if you need to do the soldermask bridge?

if not, we will do soldermask opening for the whole IC parts,ok?

If yes, we require the IC pins spacing needs to be 0.50mm with green soldermask vi,pls change it.

3. The spacing between different net about hole size should be larger than 0.3mm,pls revise the file then send the new file for check.

Really stuck on these files as i really don't know that they mean with their 3 points?

It sounds like your manufacturer's "Engineering Queries" (EQ) are pointing out some technical limitations of their production line compared to your current design.

Here is a breakdown of what these points mean and how you should address them in your PCB design software (like Altium, KiCad, or Eagle).

1. Aspect Ratio (Thickness to Hole Diameter)

The Issue: The manufacturer is concerned about the Aspect Ratio. If a board is 1.6mm thick and your drill hole is only 0.1mm, the ratio is 16:1. It is very difficult for the plating chemicals to flow through such a tiny, deep "tunnel" to coat the inside with copper.

What to do: Check your smallest via/hole size. If your board is 1.6mm thick, try to ensure no hole is smaller than 0.16mm (to keep it at 10:1).

Action: Either increase the diameter of your smallest vias or ask the manufacturer if they can handle a higher aspect ratio for an extra fee.

2. Soldermask Bridges vs. Openings

The Issue: A "soldermask bridge" is the thin strip of green mask between two SMD pads (like the pins on an IC). This bridge prevents solder from flowing between pins and causing a short circuit (bridging).

The Problem: These machines can't print a "bridge" thinner than about 0.1mm. If your IC pins are very close together, there isn't enough room for that green strip.

The Options:

Option A (Bang Mask): You remove the bridges. The entire IC area becomes one big "topping." This is riskier for soldering because bridges won't be there to stop shorts.

Option B (Modify Design): You change your footprint so the space between the copper pads is at least 0.19mm. This gives them enough room to print a reliable solder mask bridge.

Action: I recommend Option B. Check your IC footprints and ensure the gap between pads is at least 0.19mm.

- **To do:** The manufacturer wants at least **0.3mm of clearance** between the edges of any two holes that aren't connected to the same signal.
 - **Action:** Run a **Design Rule Check (DRC)** in your software with the "Hole to Hole" clearance set to 0.3mm. Move any vias or pads that are flagging as too close.

1. Adjust your via sizes (Point 1).
2. Adjust IC pad spacing (Point 2).
3. Increase clearance between holes (Point 3).
4. Export new **Gerber files** and send them the updated version.

[From: https://www.google.com/maps/@38.1616161, -78.1616161, 15z](https://www.google.com/maps/@38.1616161, -78.1616161, 15z)

[illegible]

ISSUE1 -> HOLE SIZE
The board is 1.6mm thick so all holes need to be 1.6m or greater, made this a design rule to find the 14 that were 1.27 wide and fixed them. 10 FIXED

ISSUE2->Soldermask spacing
This confuses me, as the advice says 0.19 space but our IC1 has pins 0.1 spaced apart so how is this possible?
NOT FIXED

ISSUE3-> holes cannot be 0.3 close to each other, this was made a rule and DRC ed to find all holes that were too close.
FIXED

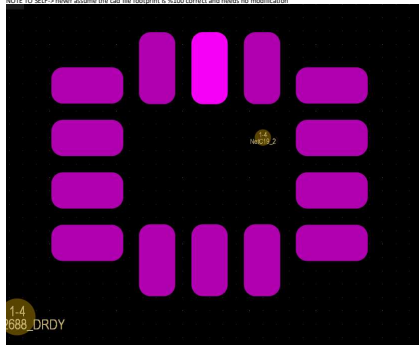
New year same project. All WIL students have lost all access so there will be a period of time where work is slowed down while access is restored. (visitor accounts were removed)
A-fix the 0.19 spacing constraint and get board shipped off
B-Wire the liscad tester board and get it shipped off aswell

notes -> 2 boards have failed to pass inspection due to issues, I don't know if both of those board were paid for, then error occurs so payment was made but the order is in limbo or payment was not made so order can be restarted
If B need to email the original order back with the new files(Brendon needs to email as I have even less access now).
If A create new order with above fixes and potentially add the tester board into the order Awe!!

thinking of waiting till the 5 day meeting to communicate this as he is very busy over the next couple days and these issues seem out of scope. Still need to ask steven about ISSUE2 as I don't know how to fix this constraint

Steven came and reviewed the issues, said this was weird and could be due to how it was inputted in pcb way. He said send it too him via email and he will see how its supposed to be inputted into pcbway (thank you steven you're the best). While we wait moving onto improving and wiring kicad tester board which is my design.

NOTE TO SELF-> never assume the cad file footprint is %100 correct and needs no modification



New pads improve
Design file below

QUT_Auto
ilot_Pixra...

Waiting for Steve to contact pcbway with a update

This is a great start for a sensor breakout and MCU test platform. The **STM32F427VIT6** is a high-performance chip, but it has very specific power requirements that must be met for it to boot and run reliably. Below is a detailed review and a *step-by-step* guide

1. Critical: VCAP Capacitors
The STM32F4 series has an internal voltage regulator that powers the CPU core. You **must** add external capacitors to the VCAP pins for the MCU to function.

- **Action:** Connect a **2.2µF** (Low ESR ceramic) capacitor from **VCAP_1** (Pin 49) to GND.
- **Action:** Connect a **2.2µF** (Low ESR ceramic) capacitor from **VCAP_2** (Pin 73) to GND.
- **Note:** Do not connect these pins to 3.3V; they are output pins for the internal regulator.

Your current schematic for U5 does not show decoupling capacitors.

- Requirement:** Every VDD pin (Pins 11, 19, 28, 50, 75, 100) and VDDA (Pin 22) needs a **0.1µF** capacitor placed as close to the pin as possible on the PCB
- Bulk Cap:** Add at least one **4.7µF** or **10µF** tantalum or ceramic capacitor to the 3.3V rail near the MCU.

3. SWD Programming Header (Missing)

While you have USB for programming via DFU mode, it is highly recommended to add an **SWD (Serial Wire Debug)** header. This allows you to use an ST-Link v2.

- Pins needed:** GND, 3.3V, SWDIO (P14 / Pin 72), SWCLK (P14 / Pin 76), and NRST (Pin 14)

4. USB ESD Protection

B. Finishing the Schematic Wiring

To finalize the board for programming and power, follow these specific wiring instructions for your switches and USB port:

- **Step 1:** Connect **U5 Pin 14 (NRST)** to one side of switch **S1**.
- **Step 2:** Connect the other side of switch **S1** to **GND**.
- **Step 3:** Add a **0.1µF** capacitor from **NRST** to **GND** (to prevent noise).
- **Step 4:** Add a **10kΩ** pull-up resistor from **NRST** to **+3.3V**.

2. Boot Switch (S2)

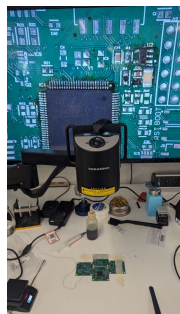
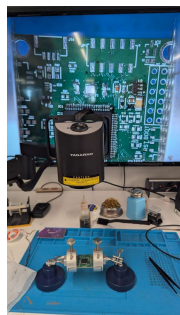
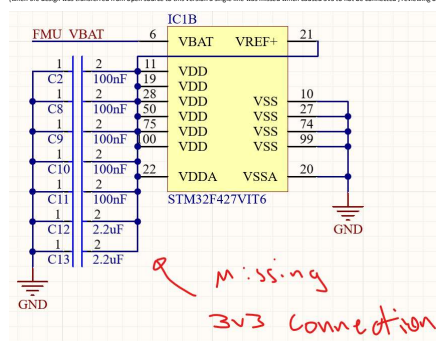
- The switch allows you to enter "DFU Mode" for programming or use.
- **Step 1:** Connect J1 Pin **(GND)** to the center pin of the 3-pin slide switch (or a push button to 3.3V).
 - **Step 2:** Connect one side of the switch to GND through a **10kΩ** resistor (Normal boot mode).
 - **Step 3:** Connect the other side of the switch to 3.3V (System Memory boot while /rebooting or Power Programming).
- **Step 4 (Optional):** To program via USB, hold the **BUTTON** switch for 30s while reading or powering on the board.
- ### 3. Micro USB (J1) & Power (UE)
- **Power (VBUS):** Connect J1 Pin 1 (VBUS) to UE Pin 1 (A1). This takes the 5V from USB and regulates it to 3.3V.
• **Data Lines –** Connect J1 Pin 2 (D+) to UE Pin 7 (PA1).
 • Connect J1 Pin 3 (D-) to UE Pin 8 (PA2).
- **ID Pin:** Leave J1 Pin 4 (ID) floating.
- **Ground:** Connect all ground pins to your common GND plane.
- **Shield:** Connect the USB housing pins to GND through a **1MΩ** resistor in parallel with a **4.7nF** capacitor (this helps with EMI/Ground loops).

Once the schematic is finished and you move back to the PCB:

1. **Ground Plane:** Use a solid ground pour on the bottom layer.
2. **Differential Pairs:** Keep the USB D+ and D- traces close together and of equal length.
3. **Sensor Placement:** You have done a good job spacing the sensors (U1-U4). Ensure the MS5611 (U4) is not placed directly under a heat source, as it is a sensitive barometric pressure sensor.

Above to be done list

Steve Advice
Ground plane on bottom, use vias to get there
2 layers
LEDs on the board for debugging





- Possible issues I can think of
 - bad physical connection on usb, stm, resistors R6 R7, Y1, C3,C4
 - stm not being powered due to jumper wire failure
 - data lines designed so poorly that the D+ D- out of sync(different impedance)
 - resistors on data line failed
 - buttons not working so board never enters DFU mode
 - Stm chip from alibaba maybe its fake and we need a new one
 - drivers or usb cord not correct(computer issue)

[illegible]

Downloading stm32cubeprogrammer and learning about dfu mode

Led red and green light programmed using simple pin high pin low code, cannot get blue for the life of me, Don't know if it's a hardware or software. Going to spend tmr learning how the led works so debugging steps make more sense

20/01/2026-

RS missing from BOM investigating
-> we have it same as R3 for now, different specs tho

I am the best to ever do it! I have my selected ICM-42688-p soldered and working on the board, led led changes colour based on different tilt angles
Use STM32_MKJDE.PRG for stm chip programming in C

going to keep populating and testing board, look at uart for testing.

OTHER SENSORS SPI NOT WORKING (could be physical soldering issue, Programming or Other like no power due to missing components)

27/01/2026-

Looking at trying to communicate with other 3 sensors on board today.

Got 2/3 working, just needed to ensure solder connections is good. Baro on the otherhand is giving me a headache. All ICB baro sensors are probs damaged and so NEED MORE IC-6. Extra extra care when soldering was needed and not done mb.

28/01/2026-
Barometer was connected to a spi2 (fram_sck etc). Once I turned on spi2 it communicated, have not tested the readings (most likely broken due to hot air gun but not important right now), stuck on Fram communication right now

moving on to Esp8266 communication for debugging as no serial wire (we could use usb but with DFU mode taking up that its gonna be a pain plus potential to damage)



Below code is sensor testing code for stm chip, requires stm32NXX to create the base for code(my input is in USER CODE 2 Section) and STM32IDE to compile to a Elf file which can then be uploaded into STM32 programmer app. Ill let you figure that out because if I could figure it out you can too <3.

Projects Page 17


```

/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_SPI1_Init();
MX_SPI2_Init();
/* USER CODE BEGIN 2 */

/* 1. SPI BUS SILENCE & POWER-UP */
// Silence SPIs (Main Sensors)
HAL_SPI_WritePola(GPIOC, ION_CS_Pins, GPIO_PIN_SET); // PCA [cite: 152]
HAL_SPI_WritePola(GPIOC, ION_CS_Pins, GPIO_PIN_SET); // PCSB [cite: 157]
HAL_SPI_WritePola(GPIOC, MAG_CS_Pins, GPIO_PIN_SET); // PESA [cite: 284]
// Silence SPI2 (Baro & FRAM) - CRITICAL: To prevent bus noise
HAL_SPI_WritePola(GPIOC, BARO_CS_Pins, GPIO_PIN_SET); // PPS [cite: 177]
HAL_SPI_WritePola(GPIOC, FRAM_CS_Pins, GPIO_PIN_SET); // PPSB [Corrected FRAM_CS]
// LEDs OFF (Active-low)
HAL_SPI_WritePola(GPIOB, GREEN_LED_Pins|RED_LED_Pins|BLUE_LED_Pins, GPIO_PIN_SET);
// Enable sensor power rails
HAL_SPI_WritePola(GPIOC, VDD_VDD_STANDBY_EN_Pins, GPIO_PIN_SET); // PE3 [cite: 233]
HAL_SPI_WritePola(GPIOC, VDD_VDD_FIRMWARE_EN_Pins, GPIO_PIN_SET); // PCA [cite: 165]
HAL_Delay(200); // Stabilization time
uint8_t id_reg, id_val;
uint8_t C[7]; // RS485 Calibration coefficients
/* --- SPI TESTS --- */
// I2C-A3888 Test
uint8_t reset_4268822[] = {0x11, 0x03}; // Soft Reset
HAL_SPI_WritePola(GPIOC, ION_CS_Pins, GPIO_PIN_RESET);
HAL_SPI_Transmit(&spi1, reset_4268822, 2, 10);
HAL_SPI_WritePola(GPIOC, ION_CS_Pins, GPIO_PIN_SET);
HAL_Delay(50);
id_reg = 0x03; id_val;
HAL_SPI_WritePola(GPIOC, ION_CS_Pins, GPIO_PIN_RESET);
HAL_SPI_Transmit(&spi1, id_reg, 1, 10);
HAL_SPI_Receive(&spi1, id_val, 1, 10);
HAL_SPI_WritePola(GPIOC, ION_CS_Pins, GPIO_PIN_SET);
if (id_val != 0x07) { while(1) { HAL_SPI_TogglePola(GPIOB, RED_LED_Pins|GREEN_LED_Pins); HAL_Delay(200); } }
// I2C-20888 Test
HAL_SPI_WritePola(GPIOC, ION_CS_Pins, GPIO_PIN_RESET);
HAL_SPI_Transmit(&spi1, id_reg, 1, 10);
HAL_SPI_Receive(&spi1, id_val, 1, 10);
if (id_val != 0x07) { while(1) { HAL_SPI_TogglePola(GPIOB, GREEN_LED_Pins|BLUE_LED_Pins); HAL_Delay(200); } }
/* 4. SENSOR 3: LIS3DH MAGNETOMETER (PE14) */
// Step A: Soft Reset & Memory
uint8_t mag_reset[2] = {0x21, 0x04}; // SOFT_RST + REBOOT [cite: 1611, 1612]
HAL_SPI_WritePola(GPIOC, MAG_CS_Pins, GPIO_PIN_RESET);
HAL_SPI_Transmit(&spi1, mag_reset, 2, 10);
HAL_SPI_WritePola(GPIOC, MAG_CS_Pins, GPIO_PIN_SET);
HAL_Delay(50);
// Step B: Wake UP - Set Operating Mode to Continuous [CRITICAL]
// CTRL_REG3 (20h), MD1=8) = 08
uint8_t mag_wakeup[2] = {0x20, 0x08};
HAL_SPI_WritePola(GPIOC, MAG_CS_Pins, GPIO_PIN_RESET);
HAL_SPI_Transmit(&spi1, mag_wakeup, 2, 10);
HAL_SPI_WritePola(GPIOC, MAG_CS_Pins, GPIO_PIN_SET);
HAL_Delay(10); // Small settle time
// Step C: Read WHO_AM_I (0Fh)
id_reg = 0x0F; id_val; // bit 0 = 1 for READ [cite: 1490]
id_val = 0;
HAL_SPI_WritePola(GPIOC, MAG_CS_Pins, GPIO_PIN_RESET);
HAL_SPI_Transmit(&spi1, id_reg, 1, 10);
HAL_SPI_Receive(&spi1, id_val, 1, 10);
HAL_SPI_WritePola(GPIOC, MAG_CS_Pins, GPIO_PIN_SET);
if (id_val != 0x0F) { // Expected value is 0x0F [cite: 1558, 1564]
    while(1) {
        HAL_SPI_TogglePola(GPIOB, RED_LED_Pins | BLUE_LED_Pins);
        HAL_Delay(200);
    }
}
/* --- SPI TESTS (BARO & FRAM) --- */
/* 5. RESISTANCE DIAGONALIZER (SPI2) */
uint8_t baro_reset = 0x1E;
HAL_SPI_WritePola(GPIOC, BARO_CS_Pins, GPIO_PIN_RESET);
HAL_SPI_Transmit(&spi2, baro_reset, 1, 10);
HAL_SPI_WritePola(GPIOC, BARO_CS_Pins, GPIO_PIN_SET);
HAL_Delay(20);
// Read FROM Calibration Coefficients C1-C6
for (uint8_t i = 1; i <= 6; i++) {
    uint8_t data[2];
    HAL_SPI_WritePola(GPIOC, BARO_CS_Pins, GPIO_PIN_RESET);
    HAL_SPI_Transmit(&spi2, i, 1, 10);
    HAL_SPI_Receive(&spi2, data, 2, 10);
    HAL_SPI_WritePola(GPIOC, BARO_CS_Pins, GPIO_PIN_SET);
    C[i] = (data[0] << 8) | data[1];
}
if (C[1] == 0 || C[1] == 0xFFFF) { HAL_SPI_WritePola(GPIOB, RED_LED_Pins, GPIO_PIN_RESET); while(1); }
/* --- RESISTANCE FRAM SYSTEM TEST (P0A8) --- */
// 1. Recovery & Backup
// Ensures the chip exits Sleep Mode (B0h) or a mid-transaction hang.
HAL_SPI_WritePola(GPIOC, FRAM_CS_Pins, GPIO_PIN_RESET);
HAL_Delay(1);
HAL_SPI_WritePola(GPIOC, FRAM_CS_Pins, GPIO_PIN_SET);
HAL_Delay(15); // B0h recovery time
// 2. Identity Verification (B010)
// Scan for Manufacturer ID (0x04) or 0x0C (Ramtron)
uint8_t rdata[12] = {0x0F};
uint8_t rdata[12] = {0};
uint8_t fram_ididentified = 0;
HAL_SPI_WritePola(GPIOC, FRAM_CS_Pins, GPIO_PIN_RESET);
HAL_SPI_TransmitReceive(&spi2, rdata, rdata, 12, 50);
HAL_SPI_WritePola(GPIOC, FRAM_CS_Pins, GPIO_PIN_SET);
for (int i = 1; i < 12; i++) {
    if (rdata[i] == 0x04 || rdata[i] == 0x0C) {
        fram_ididentified = 1;
        break;
    }
}
if (!fram_ididentified) {
    while(1) { // Error: Fast Red Flash (ID Mismatch)
        HAL_SPI_TogglePola(GPIOB, GPIO_PIN_15); // RED_LED
        HAL_Delay(50);
    }
}
// 3. Memory Integrity Check (Write/Read)
// Verify that the memory cells are non-volatile and writable.
uint8_t test_addr[2] = {0x0B, 0x01}; // Address 0x0B01
uint8_t test_data = 0x0C; // Test pattern
uint8_t read_back = 0;
uint8_t cmd_write = 0x04;
uint8_t cmd_write[4] = {0x02, test_addr[0], test_addr[1], test_data};
uint8_t cmd_read[3] = {0x03, test_addr[0], test_addr[1]};
// Write Sequence
HAL_SPI_WritePola(GPIOC, FRAM_CS_Pins, GPIO_PIN_RESET);
HAL_SPI_Transmit(&spi2, cmd_write, 4, 10);
HAL_SPI_WritePola(GPIOC, FRAM_CS_Pins, GPIO_PIN_SET);
HAL_SPI_WritePola(GPIOC, FRAM_CS_Pins, GPIO_PIN_RESET);
HAL_SPI_Transmit(&spi2, cmd_read, 3, 10);
HAL_SPI_Receive(&spi2, read_back, 1, 10);
HAL_SPI_WritePola(GPIOC, FRAM_CS_Pins, GPIO_PIN_SET);
if (read_back != test_data) {
    while(1) { // Error: Flash Red (Write/Read Failure)
        HAL_SPI_TogglePola(GPIOB, GPIO_PIN_9); // BLUE_LED
        HAL_Delay(50);
    }
}
// SUCCESS: SOLID WHITE
HAL_SPI_WritePola(GPIOB, GREEN_LED_Pins|GREEN_LED_Pins|BLUE_LED_Pins, GPIO_PIN_RESET);
HAL_Delay(200);
HAL_SPI_WritePola(GPIOB, GREEN_LED_Pins|GREEN_LED_Pins|BLUE_LED_Pins, GPIO_PIN_SET);
/* USER CODE END 3 */
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */
    /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
/**
 * Brief System Clock Configuration
 */
/*
 * @brief Name
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
    /* Configure the main internal regulator output voltage
    */
    __HAL_RCC_PMC_CLK_ENABLE();
    __HAL_RCC_VDD1V8_ANALOG12_CONFIG(RCC_REGULATOR_VOLTAGE_SCALE1);
    /** Initializes the RCC Oscillators according to the specified parameters
    * in the RCC_OscInitTypeDef structure.
    */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
    RCC_OscInitStruct.HSEState = RCC_HSE_ON;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
    RCC_OscInitStruct.PLL.PLLM = 21;
    RCC_OscInitStruct.PLL.PLLN = 180;
    RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
    RCC_OscInitStruct.PLL.PLLQ = 4;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }
    /** Activate the Over-Drive mode
    */
    if (HAL_PWREx_EnableOverDrive() != HAL_OK)
    {
        Error_Handler();
    }
    /** Initializes the CPU, AHB and APB buses clocks
    */
    RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
        |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;
    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_5) != HAL_OK)
    {
        Error_Handler();
    }
}
/**
 * Brief SPI1 Initialization Function
 */
/*
 * @brief Name
 */
static void MX_SPI1_Init(void)
{
    /* USER CODE BEGIN SPI1_Init 0 */
    /* USER CODE END SPI1_Init 0 */
    /* USER CODE BEGIN SPI1_Init 1 */
    /* USER CODE END SPI1_Init 1 */
    /* SPI parameter configuration*/
    hspi1.Instance = SPI1;
    hspi1.Init.Mode = SPI_MODE_MASTER;
    hspi1.Init.Direction = SPI_DIRECTION_2LINES;
    hspi1.Init.DataSize = SPI_DATASIZE_8BIT;
    hspi1.Init.CLKPolarity = SPI_POLARITY_HIGH;
    hspi1.Init.CLKPhase = SPI_PHASE_2EDGE;
    hspi1.Init.NSS = SPI_NSS_SOFT;
    hspi1.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_16;
    hspi1.Init.FirstSCK = SPI_FIRST_SCK;
    hspi1.Init.TIMode = SPI_TIMODE_DISABLE;
    hspi1.Init.Discontinuous = SPI_DISCONTINUATION_DISABLE;
    hspi1.Init.CRCPolynomial = 10;
    if (HAL_SPI_Init(&hspi1) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN SPI1_Init 2 */
    /* USER CODE END SPI1_Init 2 */
}
/**
 * Brief SPI2 Initialization Function
 */
/*
 * @brief Name
 */
static void MX_SPI2_Init(void)
{
    /* USER CODE BEGIN SPI2_Init 0 */
    /* USER CODE END SPI2_Init 0 */
    /* USER CODE BEGIN SPI2_Init 1 */
    /* USER CODE END SPI2_Init 1 */
    /* SPI2 parameter configuration*/
    hspi2.Instance = SPI2;
    hspi2.Init.Mode = SPI_MODE_MASTER;
    hspi2.Init.Direction = SPI_DIRECTION_2LINES;
    hspi2.Init.DataSize = SPI_DATASIZE_8BIT;
    hspi2.Init.CLKPolarity = SPI_POLARITY_HIGH;
}

```



```

    Reg12.Init.CLKPhase = SPI_PHASE_ZERO;
    Reg12.Init.NCS = SPI_NCS_S0F1;
    Reg12.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_64;
    Reg12.Init.FirstBit = SPI_FIRSTBIT_MSB;
    Reg12.Init.TIMode = SPI_TIMODE_DISABLED;
    Reg12.Init.CCSClockDivision = SPI_CCSCLOCKDIVISION_DISABLED;
    Reg12.Init.CRCPolynomial = 10;
    if (HAL_SPI_Init(&hspi2) != HAL_OK)
    {
        Error_Handler();
    }
}
/* USER CODE BEGIN SPI2_Init 2 */
/* USER CODE END SPI2_Init 2 */

/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    /* USER CODE BEGIN MX_GPIO_Init_1 */
    /* USER CODE END MX_GPIO_Init_1 */
    /* GPIO Ports Clock Enable */
    HAL_RCC_GPIOC_CLK_ENABLE();
    HAL_RCC_GPIOC_CLK_ENABLE();
    HAL_RCC_GPIOC_CLK_ENABLE();
    HAL_RCC_GPIOC_CLK_ENABLE();
    HAL_RCC_GPIOC_CLK_ENABLE();
    HAL_RCC_GPIOC_CLK_ENABLE();
    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOE, VDD_3V3_SENSORS_EN_Pin|WAKE_CS_Pin, GPIO_PIN_SET);
    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOC, IO4_CS_Pin|IO4_CS_Pin|VDD_3V3_PERIPH_EN_Pin, GPIO_PIN_SET);
    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOB, GREEN_LED_Pin|RED_LED_Pin|BLUE_LED_Pin, GPIO_PIN_SET);
    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOA, FRAM_CS_Pin|BARO_CS_Pin, GPIO_PIN_SET);
    /*Configure GPIO pins : VDD_3V3_SENSORS_EN_Pin WAKE_CS_Pin */
    GPIO_InitStruct.Pin = VDD_3V3_SENSORS_EN_Pin|WAKE_CS_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOE, &GPIO_InitStruct);
    /*Configure GPIO pins : IO4_CS_Pin|IO4_CS_Pin|VDD_3V3_PERIPH_EN_Pin */
    GPIO_InitStruct.Pin = IO4_CS_Pin|IO4_CS_Pin|VDD_3V3_PERIPH_EN_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);
    /*Configure GPIO pins : GREEN_LED_Pin|RED_LED_Pin|BLUE_LED_Pin */
    GPIO_InitStruct.Pin = GREEN_LED_Pin|RED_LED_Pin|BLUE_LED_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
    /*Configure GPIO pins : FRAM_CS_Pin|BARO_CS_Pin */
    GPIO_InitStruct.Pin = FRAM_CS_Pin|BARO_CS_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
    /* USER CODE BEGIN MX_GPIO_Init_2 */
    /* USER CODE END MX_GPIO_Init_2 */
}
/* USER CODE BEGIN 4 */
/* USER CODE END 4 */

/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return state */
    while(1)
    {
        /* USER CODE END Error_Handler_Debug */
    }
}

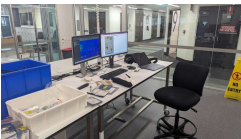
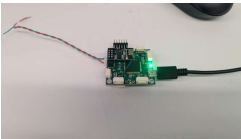
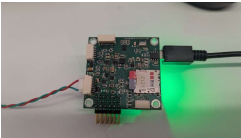
#ifdef USE_FULL_ASSERT
/**
 * @brief Report the name of the source file and the source line number
 * @param where: where the assert_param error has occurred
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* User can add his own implementation to report the file name and line number,
     * ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
    /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */

```

moving into esp testing, need to write/ find simple code for the actual esp then check esp-nvm connection then look at reading esp base station using Arduino or something idk

02/02/2026-

V2 board fabricated, lights are white so all key sensors com. Looks good very happi



28/01/2026-

I sent 12.5 Volts into board and it caught fire.

Had to replace all regulators, IC11 U1 U2. Seemed to fix got off lucky. Stupid as ESC breakout didn't come with BEC and the V is 52 like wtf

02/02/2026-

Finished all fabrication of the board and moved into starting firmware programming. Need to roll openmorse PXT and add main68268 to smali will chip on board. Using my arch linux computer for this, need the ps4 github repo/mods/drivers and get Chip select in board config, the main68266 firmware repo. Lots of issues with this so take your time solving them if repeating.

03/02/2026-

Started with mavlink266 firmware, i got the px4 firmware loaded but no sensors were reading so i stopped. i got a ESP-USB-SERIAL github repo and followed the instructions to turn a small esp32c3 into a usb-serial converter, then used platform io and my computer to flash the esp-02_1M mavlink software onto the wifi chip. Was a pain but this worked well once we regain in the air. Before that i spent time rigging apart a small racing drone i owned (same compo that send 12V into my board) to use as a small testing rig for external batt power and basic motor control. No props and need to upgrade but it was a good small platform for initial testing, no radio need to steal one from cleve

09/02/2026-

Have looked into this firmware, spent a lot of the day going WTF but using gemini realised that i had only added the new sensor drivers and the CS pins were not updated in firmware(this was mentioned above but im writing all of this now so hindsight's annoying). After modifying and re-flashing MAKE SURE THE CHIP HAS THE BOOTLOADER INSTALLED OTHERWISE PX4 FLASH DONT WORK!! i got sensors to read, ids about accuracy or errors but we can fix that up later. Wanted wifi now since i did all that effort above, Yes an MAVLINK_0 to wifi port on QGroundControl then go to serial and set wifi serial to 920000ish then it all works perfectly(had to figure that out the hard way), noticed that imu data is not transferred wirelessly. Could control motor outputs on the board wirelessly from ground control which was a huge success, i have a video but that vid files would prob get test. Moving into calibrating sensors correctly, connecting antenna so Qground/control errors would go away. Once they are all gone may speak to cleve about drone flying using board(we will break a drone probs)

10/02/2026-

Spent this time preparing for the first flight, was given access to a large holy bro drone and added my board to the drone. Also tested mavlink wifi for the board and added a RC connection so the pilot had proper equipment for test. Couldn't test the props on battery indoor without the pilot and safety net so after tests complete we were done for the day.

20/02/2026-

TESTDAY, wrote up a meagu document slides for the pilot. These detailed the board specs and changes. At 1 we went to level 1 oblock and met the pilot (the goat)

Julian Galvez Serna

He configured the drone and ran some initial tests that standard pilots do. This was then looking at battery install and prop testing (M3MA had to be switched thank you Julian for the pickup). Then moved the drone into the netted area and performed Maiden flight for the board, all tests were green and there was some sway in the idle hover(manual mode). Very happy !

Moving to adding a companion computer and uart telemetry 1 connection so auto pilot mode can be tested(using inside camera tracking system to map the drones position and using these position values to tune the autopilot mode)

24/02/2026-

I have added a pins with the custom power board. The Fmu board was attached above and a custom USB-uart converter was made using a esp32-S3 to break out into a XTEN connector for TSLIM. This was tested with a ros2 download and the mavros node. Ros2 was installed Using Tobias's ROBOTSTACK approach and the result showed that ros2 could visualize the imu output. This proved that the Fmu board could connect to a companion computer. Testing of this was to occur the next day

SECOND FLIGHT TEST

The second flight test tested sensor fusion using a indoor tracking system and a companion computer. Due to the previous tests all that was need to be modified was the ACMD and baudrate. The first issue was there was large error between the indoor position readings and the Accelerometer position readings. This was fixed by turning on a EKF setting. The Height was then taking this was fixed by turning off the barometer and only using indoor position data. The drone had some issue with yaw drift however this could have been due to a poor weight distribution on the airframe. The first test was a position hold test using the indoor tracking system and that worked. When moving to the final waypoint test a error which stopped offboard mode from activating occurred every time during flight. No cause was identified and the test concluded. Waypoint mode would have worked ideally if offboarding mode could be activated mid flight, this also may have been linked to the RC mode switching not working aswell. These 2 firmware issues are the key issues to fix in code before the board can move on to next phase(accuracy definitions or something in that scope)

27/02/26-

Compiling files for handover end of this log as no new process being made

